FULL BUILD PROMPT FOR AI AGENT

VIGILANTE

Resume Tailor

"Your resume, tailored for the job, AI-powered and ready for ATS."

Audience	Phases	Stack
AI Build Agent	**7 Phases**	**Next.js + Supabase + Gemini**

0. How To Read & Execute This Prompt

MANDATORY — Agent Instructions Before Starting

This is a complete, authoritative build specification. Read the entire document before writing any code.

- Execute phases in strict order (Phase 0 → Phase 1 → … → Phase 6). Never begin a phase before completing and verifying the previous one.
- After each phase, write a phase report to /reports/phase-N-report.md before proceeding.
- Every rule marked [MUST] is non-negotiable. Rules marked [SHOULD] are strong defaults.
- When the user must perform a manual action (Supabase setup, filling .env.local, etc.), PAUSE and output a clearly formatted 'ACTION REQUIRED' block. Wait for confirmation before continuing.
- Never hardcode secrets, API keys, or credentials. Always use process.env.*.
- All source code must be TypeScript with strict mode enabled. No 'any' types.
- Test coverage must be written alongside feature code, not after.
- The design system from the UI Design Prompt (already implemented by the design agent) must be respected. Do not reinvent UI components.

1. Project Context & Goals

VIGILANTE Resume Tailor is a production-grade, AI-powered fullstack web application. It allows authenticated users to upload a resume (PDF/DOCX or plain text), input a job description (text or URL), and receive a tailored, ATS-optimized resume generated by Gemini AI. The tailored resume is previewed in-app, available for PDF download, and auto-sent to the user's email.

The application supports English and Arabic with full RTL layout. All user data is stored securely in Supabase (PostgreSQL + Storage). The system enforces rate limiting, input validation, error monitoring, and privacy-first data handling.

1.1 Non-Goals (Excluded from This Build)

- No multi-tenant SaaS billing, subscription tiers, or payment processing.
- No Chrome extension or browser plugin.
- No job application tracker or status board.
- No LinkedIn API integration or OAuth social login.
- No team/org accounts or shared workspaces.
- No mobile native app (iOS/Android).

2. Final Tech Stack

Layer	Tool / Library	Version / Notes
Framework	Next.js 14	App Router, TypeScript, strict mode
Language	TypeScript	5.x, strict: true, no implicit any
Styling	TailwindCSS + shadcn/ui	Design tokens from UI Design Prompt
Theme	next-themes	Dark default, light mode supported
i18n	next-intl	EN + AR, RTL via dir attribute on <html>
Auth	Supabase Auth	Email/password, JWT sessions, middleware protection
Database	Supabase PostgreSQL	Row-Level Security on all tables
File Storage	Supabase Storage	Signed URLs, 5MB limit, PDF + DOCX
AI	Google Gemini API	gemini-1.5-flash model, structured JSON output
Resume Parse (PDF)	pdf-parse	Server-side only, validates text length
Resume Parse (DOCX)	mammoth	Server-side only, extracts raw text
Job URL Scraping	cheerio + node-fetch	Sanitized, timeout-protected
PDF Generation	@react-pdf/renderer	ATS-safe, 3 templates, RTL support

Email Delivery	Resend SDK	resend.com, SPF/DKIM configured, PDF attachment
Validation	Zod	All API route inputs and AI output schemas
Rate Limiting	@upstash/ratelimit + @upstash/redis	Per-user-id, 5 tailorings/day
Forms	react-hook-form + @hookform/resolvers	Zod resolvers
Data Fetching	@tanstack/react-query	Loading, error, cache, refetch states
Animations	framer-motion	Result card, score ring, stagger reveals
Toasts	sonner	4 toast variants as per design spec
Date Formatting	date-fns	EN + AR locale formatting
Error Monitoring	Sentry.io	Frontend + API route errors, free tier
Analytics	Posthog	Event tracking, funnel analysis, free tier
Testing (Unit)	Vitest	Fast, native TypeScript, ESM
Testing (Integration)	@testing-library/react	Component + page tests
Testing (E2E)	Playwright	Critical user flows
Linting	ESLint + eslint-config-next	Strict ruleset
Formatting	Prettier	Consistent code style
Deployment	Vercel	Preview + Production environments
CI	GitHub Actions	Lint + type-check + test on every push

3. Required Project Structure

The agent MUST create this exact folder and file structure. Every directory listed must exist. Files marked with ← must be created with real content, not empty placeholders.

```
vigilante-resume-tailor/
├── .github/
│   └── workflows/
│       └── ci.yml                  ← GitHub Actions: lint, typecheck, test
├── docs/
│   ├── requirements.md             ← Written by agent at Phase 0 start
│   ├── walkthrough.md              ← Written by agent at Phase 4 end
│   └── final.md                    ← Written by agent at Phase 6 end
├── reports/
│   ├── phase-0-report.md           ← Written at end of each phase
│   ├── phase-1-report.md
│   ├── phase-2-report.md
```

```
│   ├── phase-3-report.md
│   ├── phase-4-report.md
│   ├── phase-5-report.md
│   └── phase-6-report.md
├── tests/
│   ├── unit/
│   │   ├── resume-parser.test.ts   ← pdf-parse + mammoth logic
│   │   ├── gemini-prompt.test.ts   ← Prompt construction + output schema
│   │   ├── pdf-generator.test.ts   ← PDF template rendering
│   │   ├── ats-scorer.test.ts      ← Keyword matching logic
│   │   ├── rate-limiter.test.ts    ← Upstash rate limit logic
│   │   └── zod-schemas.test.ts     ← All Zod schema validations
│   ├── integration/
│   │   ├── api-tailor.test.ts      ← /api/tailor full pipeline test
│   │   ├── api-parse.test.ts       ← /api/parse-resume tests
│   │   ├── api-export.test.ts      ← /api/export-pdf tests
│   │   ├── api-email.test.ts       ← /api/send-email tests
│   │   └── supabase-rls.test.ts    ← RLS policy verification tests
│   ├── e2e/
│   │   ├── auth.spec.ts            ← Sign up, login, logout
│   │   ├── tailoring-flow.spec.ts  ← Full tailor → download flow
│   │   ├── history.spec.ts         ← History list + detail
│   │   ├── settings.spec.ts        ← Language switch, email toggle
│   │   └── rtl.spec.ts             ← Arabic RTL layout verification
│   └── setup/
│       ├── vitest.config.ts
│       ├── playwright.config.ts
│       └── test-utils.tsx          ← Custom render wrappers
├── src/
│   ├── app/                        ← Next.js App Router pages
│   │   ├── layout.tsx
│   │   ├── globals.css
│   │   ├── page.tsx                ← Landing
│   │   ├── auth/login/page.tsx
│   │   ├── auth/signup/page.tsx
│   │   ├── dashboard/layout.tsx    ← App shell
│   │   ├── dashboard/page.tsx      ← Main tailor page
│   │   ├── dashboard/history/page.tsx
│   │   ├── dashboard/history/[id]/page.tsx
│   │   ├── dashboard/settings/page.tsx
│   │   └── api/
│   │       ├── tailor/route.ts
│   │       ├── parse-resume/route.ts
│   │       ├── fetch-job/route.ts
│   │       ├── export-pdf/route.ts
│   │       └── send-email/route.ts
│   ├── components/
│   │   ├── ui/                     ← shadcn/ui primitives
│   │   └── app/                    ← Feature components (from UI prompt)
│   ├── lib/
│   │   ├── supabase/
│   │   │   ├── client.ts           ← Browser Supabase client
│   │   │   ├── server.ts           ← Server Supabase client
│   │   │   └── middleware.ts       ← Session refresh middleware
│   │   ├── gemini/
│   │   │   ├── client.ts           ← Gemini SDK init
│   │   │   ├── prompt.ts           ← Prompt builder (versioned)
```

```
│   │   │   └── schema.ts          ← Zod schema for AI output
│   │   ├── pdf/
│   │   │   ├── generator.ts       ← @react-pdf/renderer logic
│   │   │   ├── templates/
│   │   │   │   ├── classic.tsx
│   │   │   │   ├── modern.tsx
│   │   │   │   └── arabic-rtl.tsx
│   │   ├── parser/
│   │   │   ├── pdf-parser.ts      ← pdf-parse wrapper
│   │   │   └── docx-parser.ts     ← mammoth wrapper
│   │   ├── scraper/
│   │   │   └── job-scraper.ts     ← cheerio job URL fetcher
│   │   ├── email/
│   │   │   └── sender.ts          ← Resend SDK wrapper
│   │   ├── rate-limit/
│   │   │   └── limiter.ts         ← Upstash rate limit helper
│   │   ├── security/
│   │   │   └── sanitize.ts        ← Input sanitization utilities
│   │   ├── ats/
│   │   │   └── scorer.ts          ← Keyword extraction + ATS score
│   │   ├── design-tokens.ts
│   │   └── utils.ts
│   ├── hooks/
│   │   ├── useLanguage.ts
│   │   ├── useTheme.ts
│   │   └── useTailoring.ts        ← React Query mutation + state
│   ├── types/
│   │   ├── resume.ts              ← Resume, TailoredResume, etc.
│   │   ├── api.ts                 ← API request/response types
│   │   └── supabase.ts            ← Generated Supabase DB types
│   └── messages/
│       ├── en.json
│       └── ar.json
├── supabase/
│   ├── migrations/
│   │   ├── 001_create_profiles.sql
│   │   ├── 002_create_resumes.sql
│   │   └── 003_rls_policies.sql
│   └── seed.sql                   ← Optional dev seed data
├── .env.local                     ← Created by agent, user fills values
├── .env.example                   ← Safe copy with placeholder values
├── .gitignore
├── next.config.ts
├── tailwind.config.ts
├── tsconfig.json                  ← strict: true
├── package.json
└── README.md
```

4. Environment Variables & External Setup

4.1 .env.local — Create This File Immediately at Phase 0

The agent must create .env.local with all keys present but empty (or with placeholder values where noted). Output an ACTION REQUIRED block for the user to fill in each value.

```
# ──────────────────────────────────────────────────────
# VIGILANTE Resume Tailor — Environment Variables
# Fill ALL values before running the application.
# NEVER commit this file to version control.
# ──────────────────────────────────────────────────────

# ── Supabase ──────────────────────────────────────
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

# ── Google Gemini AI ──────────────────────────────
GEMINI_API_KEY=

# ── Resend (Email Delivery) ───────────────────────
RESEND_API_KEY=
RESEND_FROM_EMAIL=noreply@yourdomain.com

# ── Upstash Redis (Rate Limiting) ─────────────────
UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=

# ── Sentry (Error Monitoring) ─────────────────────
NEXT_PUBLIC_SENTRY_DSN=
SENTRY_AUTH_TOKEN=
SENTRY_ORG=
SENTRY_PROJECT=

# ── Posthog (Analytics) ───────────────────────────
NEXT_PUBLIC_POSTHOG_KEY=
NEXT_PUBLIC_POSTHOG_HOST=https://app.posthog.com

# ── App Config ────────────────────────────────────
NEXT_PUBLIC_APP_URL=http://localhost:3000
NEXT_PUBLIC_APP_NAME=VIGILANTE Resume Tailor
RATE_LIMIT_MAX_PER_DAY=5
MAX_FILE_SIZE_BYTES=5242880
```

4.2 User Setup Guide — Supabase

ACTION REQUIRED — Supabase Setup (User Must Complete)

The agent must output these exact steps and wait for the user to confirm completion before running migrations.

STEP 1: Go to https://supabase.com → Create a new project.

Choose a strong database password. Save it securely.

STEP 2: Copy your credentials from Project Settings → API:
- Project URL → paste into NEXT_PUBLIC_SUPABASE_URL
- anon (public) key → paste into NEXT_PUBLIC_SUPABASE_ANON_KEY
- service_role (secret) key → paste into SUPABASE_SERVICE_ROLE_KEY

⚠ The service_role key has full DB access. Never expose it client-side.

STEP 3: In Supabase Dashboard → Storage → Create a new bucket named: resumes
Set it to PRIVATE (not public). This is critical for security.

STEP 4: In Supabase Dashboard → Authentication → URL Configuration:
- Site URL: http://localhost:3000 (change to production URL after deploy)
- Redirect URLs: http://localhost:3000/auth/callback

STEP 5: Enable Email provider under Authentication → Providers → Email.
Enable 'Confirm email' toggle.

STEP 6: Confirm with the agent when all 5 steps are done.

4.3 User Setup Guide — Resend

ACTION REQUIRED — Resend Email Setup

STEP 1: Go to https://resend.com → Create a free account.
STEP 2: Add and verify your sending domain (even a free subdomain works).
Follow their DNS records guide (adds SPF, DKIM, DMARC records to your domain).
STEP 3: Create an API key → paste into RESEND_API_KEY.
STEP 4: Set RESEND_FROM_EMAIL to a verified sender address on your domain.
e.g. resume@yourdomain.com

4.4 User Setup Guide — Upstash Redis

ACTION REQUIRED — Upstash Redis Setup

STEP 1: Go to https://upstash.com → Create a free account.
STEP 2: Create a new Redis database → choose the region closest to your Vercel deployment.
STEP 3: From the database dashboard, copy:
- REST URL → paste into UPSTASH_REDIS_REST_URL
- REST Token → paste into UPSTASH_REDIS_REST_TOKEN

4.5 User Setup Guide — Gemini API

ACTION REQUIRED — Google Gemini API Key

STEP 1: Go to https://aistudio.google.com/app/apikey
STEP 2: Click 'Create API Key' → select a Google Cloud project (or create one).
STEP 3: Copy the API key → paste into GEMINI_API_KEY.
The free tier includes: 15 RPM, 1 million tokens/min, 1500 requests/day (Gemini 1.5 Flash).
This is sufficient for personal/small-scale use.

4.6 User Setup Guide — Sentry & Posthog

ACTION REQUIRED — Monitoring & Analytics

Sentry:
STEP 1: Go to https://sentry.io → Create a free account.
STEP 2: Create a new project → choose Next.js.
STEP 3: Copy DSN → NEXT_PUBLIC_SENTRY_DSN. Copy auth token → SENTRY_AUTH_TOKEN.
STEP 4: Set SENTRY_ORG and SENTRY_PROJECT to your org/project slug from Sentry.

Posthog:
STEP 1: Go to https://posthog.com → Create a free account.
STEP 2: Create a new project → copy the Project API Key → NEXT_PUBLIC_POSTHOG_KEY.

5. Database Schema & Security

5.1 Migration: 001_create_profiles.sql

```
-- Enable UUID extension
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

CREATE TABLE public.profiles (
  id                    UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE
CASCADE,
  email                 TEXT NOT NULL,
  full_name             TEXT,
  language              TEXT NOT NULL DEFAULT 'en' CHECK (language IN
('en','ar')),
  default_resume_lang   TEXT NOT NULL DEFAULT 'en' CHECK (default_resume_lang
IN ('en','ar')),
  email_notifications   BOOLEAN NOT NULL DEFAULT TRUE,
  daily_tailoring_count INTEGER NOT NULL DEFAULT 0,
  last_tailoring_date   DATE,
  created_at            TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
  updated_at            TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Auto-update updated_at
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN NEW.updated_at = NOW(); RETURN NEW; END;
$$ language 'plpgsql';

CREATE TRIGGER update_profiles_updated_at
  BEFORE UPDATE ON public.profiles
  FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

-- Auto-create profile on user sign-up
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.profiles (id, email, full_name)
  VALUES (
    NEW.id,
    NEW.email,
    COALESCE(NEW.raw_user_meta_data->>'full_name', '')
  );
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

5.2 Migration: 002_create_resumes.sql

```
CREATE TABLE public.resumes (
  id                  UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id             UUID NOT NULL REFERENCES public.profiles(id) ON DELETE
CASCADE,
  original_filename   TEXT,
  original_text       TEXT NOT NULL,
  original_file_url   TEXT,
  job_description     TEXT NOT NULL,
  job_title           TEXT,
  company_name        TEXT,
  job_source_url      TEXT,
  tailored_json       JSONB NOT NULL,
  tailored_pdf_url    TEXT,
  ats_score           INTEGER CHECK (ats_score >= 0 AND ats_score <= 100),
  keywords_matched    TEXT[] DEFAULT '{}',
  keywords_missing    TEXT[] DEFAULT '{}',
  language            TEXT NOT NULL DEFAULT 'en' CHECK (language IN
('en','ar')),
  tailoring_strength  TEXT NOT NULL DEFAULT 'balanced' CHECK
(tailoring_strength IN ('conservative','balanced','aggressive')),
```

```
  template_used       TEXT NOT NULL DEFAULT 'classic' CHECK (template_used IN
('classic','modern','arabic-rtl')),
  email_sent          BOOLEAN NOT NULL DEFAULT FALSE,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_resumes_user_id ON public.resumes(user_id);
CREATE INDEX idx_resumes_created_at ON public.resumes(created_at DESC);
CREATE INDEX idx_resumes_user_created ON public.resumes(user_id, created_at
DESC);
```

5.3 Migration: 003_rls_policies.sql

```
-- ── Profiles RLS ──────────────────────────────────────────────
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

-- Users can only read their own profile
CREATE POLICY "profiles_select_own" ON public.profiles
  FOR SELECT USING (auth.uid() = id);

-- Users can only update their own profile
CREATE POLICY "profiles_update_own" ON public.profiles
  FOR UPDATE USING (auth.uid() = id);

-- Insert is handled by the trigger (service role), not users directly
CREATE POLICY "profiles_insert_service" ON public.profiles
  FOR INSERT WITH CHECK (auth.uid() = id);

-- ── Resumes RLS ───────────────────────────────────────────────
ALTER TABLE public.resumes ENABLE ROW LEVEL SECURITY;

CREATE POLICY "resumes_select_own" ON public.resumes
  FOR SELECT USING (auth.uid() = user_id);

CREATE POLICY "resumes_insert_own" ON public.resumes
  FOR INSERT WITH CHECK (auth.uid() = user_id);

CREATE POLICY "resumes_update_own" ON public.resumes
  FOR UPDATE USING (auth.uid() = user_id);

CREATE POLICY "resumes_delete_own" ON public.resumes
  FOR DELETE USING (auth.uid() = user_id);

-- ── Storage RLS ───────────────────────────────────────────────
-- In Supabase Storage, the bucket 'resumes' must be PRIVATE.
-- Path convention: resumes/{user_id}/originals/{filename}
--                  resumes/{user_id}/tailored/{resume_id}.pdf

CREATE POLICY "storage_select_own" ON storage.objects
  FOR SELECT USING (
    bucket_id = 'resumes' AND
    (storage.foldername(name))[1] = auth.uid()::text
```

```
  );

CREATE POLICY "storage_insert_own" ON storage.objects
  FOR INSERT WITH CHECK (
    bucket_id = 'resumes' AND
    (storage.foldername(name))[1] = auth.uid()::text
  );

CREATE POLICY "storage_delete_own" ON storage.objects
  FOR DELETE USING (
    bucket_id = 'resumes' AND
    (storage.foldername(name))[1] = auth.uid()::text
  );
```

RLS Verification Requirement

After applying migrations, the agent MUST verify RLS is working correctly by:
1. Running the supabase-rls.test.ts integration test suite.
2. Attempting a cross-user data access query in the Supabase SQL editor — it must return 0 rows.
3. Confirming that the Storage bucket 'resumes' is set to PRIVATE in the Supabase dashboard.
DO NOT proceed to Phase 2 until RLS tests pass.

6. Gemini AI Prompt Architecture (Versioned)

The Gemini prompt is versioned code. It lives in src/lib/gemini/prompt.ts and is the core IP of the product. It must be treated with the same rigor as any other module — typed, tested, and iterable.

6.1 System Prompt (Exact Text)

```
export const PROMPT_VERSION = '1.0.0';

export function buildSystemPrompt(): string {
  return `You are an expert resume writer and ATS (Applicant Tracking System)
optimization specialist.
Your task is to rewrite the provided resume to maximize its match with the
given job description.

=== ABSOLUTE RULES — NEVER VIOLATE ===
1. NEVER invent, fabricate, add, or embellish any skill, tool, experience,
qualification,
   certification, company, date, or metric not explicitly stated in the
original resume.
2. You MAY rephrase, reorder, and reframe existing content to better align with
job language.
```

```
3. You MAY replace vague terms with the job description's exact vocabulary if
semantically identical.
   Example: 'worked with teams' → 'cross-functional collaboration' (only if
context matches).
4. Prioritize bullet points that are quantifiable and outcome-driven.
5. NEVER add a skills or tool to the skills section that is not in the original
resume.
6. Output ONLY valid JSON matching the provided schema. No markdown, no prose,
no explanation.

=== ATS OPTIMIZATION RULES ===
7. Match the job description's keyword vocabulary precisely (exact phrases over
synonyms).
8. Place the most job-relevant experience and skills sections first.
9. Remove or demote experience sections irrelevant to this role.
10. Ensure the professional summary (2-3 sentences) directly addresses the
job's core requirements.

=== LANGUAGE RULES ===
11. Output language: {LANGUAGE}. If Arabic, use formal Modern Standard Arabic
(MSA).
12. If Arabic: all text must be grammatically correct RTL Arabic. Do not mix
EN/AR in same field.

=== TAILORING STRENGTH ===
13. Strength level: {STRENGTH}.
    conservative = minimal rewrites, preserve original voice, only add missing
keywords organically.
    balanced = moderate rewrites, strong keyword alignment, restructure bullets
for impact.
    aggressive = maximum keyword matching, full rewrite of all bullets for ATS,
bold restructuring.
  `
}
```

6.2 User Prompt Builder

```
export function buildUserPrompt(
  originalText: string,
  jobDescription: string,
  language: 'en' | 'ar',
  strength: 'conservative' | 'balanced' | 'aggressive'
): string {
  return `
[ORIGINAL RESUME]
${originalText}

[JOB DESCRIPTION]
${jobDescription}

[INSTRUCTIONS]
Language: ${language === 'en' ? 'English' : 'Arabic'}
Tailoring Strength: ${strength}
```

```
Return ONLY a JSON object matching this exact schema:
${JSON.stringify(TAILORED_RESUME_SCHEMA_EXAMPLE, null, 2)}
  `
}
```

6.3 Output Zod Schema (src/lib/gemini/schema.ts)

```
import { z } from 'zod';

export const ExperienceItemSchema = z.object({
  company:   z.string(),
  title:     z.string(),
  location:  z.string().optional(),
  startDate: z.string(),
  endDate:   z.string(),
  bullets:   z.array(z.string()).min(1).max(8),
});

export const EducationItemSchema = z.object({
  institution: z.string(),
  degree:      z.string(),
  field:       z.string().optional(),
  startDate:   z.string(),
  endDate:     z.string(),
  gpa:         z.string().optional(),
});

export const CertificationItemSchema = z.object({
  name:   z.string(),
  issuer: z.string(),
  date:   z.string().optional(),
});

export const TailoredResumeSchema = z.object({
  contact: z.object({
    name:     z.string(),
    email:    z.string().email().optional(),
    phone:    z.string().optional(),
    location: z.string().optional(),
    linkedin: z.string().url().optional(),
    website:  z.string().url().optional(),
  }),
  summary:          z.string().min(20).max(500),
  experience:       z.array(ExperienceItemSchema),
  education:        z.array(EducationItemSchema),
  skills:           z.array(z.string()),
  certifications:   z.array(CertificationItemSchema).optional(),
  keywords_matched: z.array(z.string()),
  keywords_missing: z.array(z.string()),
  ats_score:        z.number().int().min(0).max(100),
  prompt_version:   z.string(),
});
```

```
export type TailoredResume = z.infer<typeof TailoredResumeSchema>;
```

7. API Routes — Complete Specifications

7.1 POST /api/parse-resume

Aspect	Specification
Input	FormData: file (PDF or DOCX, max 5MB)
Auth	Required — validate Supabase session server-side
File Validation	MIME type check (application/pdf | application/vnd.openxmlformats...). Reject all others with 400.
Size Validation	Reject files > MAX_FILE_SIZE_BYTES with 413.
PDF Parsing	pdf-parse. If extracted text < 200 chars, return 422 with: { error: 'SCANNED_PDF', message: 'Could not extract text. Please paste your resume text instead.' }
DOCX Parsing	mammoth.extractRawText(). Same 200-char minimum check.
Output	{ text: string, charCount: number, detectedLanguage: 'en' | 'ar' }
Language Detection	Simple heuristic: if > 30% of unique characters are Arabic Unicode range (0x0600–0x06FF), return 'ar'. Else 'en'.
Error Handling	All errors return { error: string, message: string } with appropriate HTTP status.

7.2 POST /api/fetch-job

Aspect	Specification
Input	{ url: string } — Zod validated (must be valid HTTPS URL)
Auth	Required
Allowed Domains	No domain allowlist — but must be HTTPS only. Reject HTTP with 400.
Fetch	node-fetch with timeout: 8000ms. Reject on timeout with 408.
Extraction	cheerio: extract text from <main>, <article>, .job-description, .description (in priority order). Strip all HTML tags.
Sanitization	Run through sanitize.ts: remove script/style tags, control chars, limit to 8000 chars.

Output	{ jobDescription: string, charCount: number }
Prompt Injection Guard	Scan extracted text for patterns matching system prompt injection (e.g., 'ignore previous', 'you are now', 'SYSTEM:'). If found, return 422 with INJECTION_DETECTED error.

7.3 POST /api/tailor

Aspect	Specification
Input	{ originalText, jobDescription, language, strength, template } — all Zod validated
Auth	Required
Rate Limiting	Check Upstash: user's daily_tailoring_count. If >= RATE_LIMIT_MAX_PER_DAY, return 429 with { error: 'RATE_LIMIT', resetsAt: ISO timestamp of midnight }.
Input Lengths	originalText: 200–15000 chars. jobDescription: 100–8000 chars. Reject out of range.
Gemini Call	Call with buildSystemPrompt() + buildUserPrompt(). Model: gemini-1.5-flash. Max tokens: 4096. Temperature: 0.3.
Output Parsing	Parse JSON from Gemini response. Validate against TailoredResumeSchema. If validation fails, retry once with a stricter prompt. If second attempt fails, return 502.
DB Write	Insert into resumes table with all fields. Increment daily_tailoring_count in profiles.
Storage	Save originalText to Supabase Storage at resumes/{user_id}/originals/{uuid}.txt if no file was uploaded.
Output	{ resumeId: string, tailored: TailoredResume, atsScore: number }
Timeout	Set Vercel function timeout to 60s in next.config.ts for this route.

7.4 POST /api/export-pdf

Aspect	Specification
Input	{ resumeId: string, template: 'classic' | 'modern' | 'arabic-rtl' }
Auth	Required. Verify that the resumeId belongs to the authenticated user (RLS + explicit check).
PDF Generation	Fetch tailored_json from DB. Pass to the appropriate @react-pdf/renderer template. Generate buffer.
Storage	Upload PDF buffer to Supabase Storage: resumes/{user_id}/tailored/{resumeId}.pdf with signed URL (1 hour expiry). Update tailored_pdf_url in resumes table.

Output	{ downloadUrl: string } — the signed URL
Content-Type	Return PDF directly as application/pdf with appropriate headers if called with ?inline=true

7.5 POST /api/send-email

Aspect	Specification
Input	{ resumeId: string }
Auth	Required. Verify resumeId ownership.
PDF Fetch	Download the tailored PDF from Supabase Storage. If not yet generated, call export-pdf logic first.
Email	Send via Resend SDK. To: user's email (from profiles). Attachment: PDF buffer. Subject: 'Your Tailored Resume — [job_title] at [company_name]'. HTML body: branded template with download link.
DB Update	Set email_sent = true on the resume record.
Idempotency	If email_sent is already true, still send (user may want it re-sent) but note it in the response.
Output	{ success: true, sentTo: string }

8. Security Requirements — Mandatory

[MUST] — All security requirements are non-negotiable
These are not optional hardening steps. Every item must be implemented before Phase 5.

Requirement	Implementation
Auth Middleware	src/middleware.ts: use Supabase session refresh on every request. Redirect unauthenticated users from /dashboard/* to /auth/login.
Server-only secrets	SUPABASE_SERVICE_ROLE_KEY, GEMINI_API_KEY, RESEND_API_KEY, UPSTASH tokens — NEVER import in client components. Only in server-side API routes.
.gitignore	.env.local must be in .gitignore from project init. .env.example must contain only placeholder values.
File Upload Security	Validate MIME type server-side (not just extension). Use file-type library for magic byte detection. Reject files not matching application/pdf or OOXML MIME types.

Input Sanitization	All user text inputs pass through sanitize.ts before use: strip null bytes, control chars, excessive whitespace. Job description max 8000 chars enforced at API layer.
Prompt Injection Guard	Implemented in /api/fetch-job and /api/tailor: scan inputs for injection patterns before sending to Gemini.
Supabase Signed URLs	ALL file access via signed URLs with max 1-hour expiry. No public file URLs anywhere in the codebase.
RLS Verification	All DB tables have RLS enabled. Verified by automated tests. No service role key used in client-facing code.
Rate Limiting	Upstash Redis per-user-id rate limiting. Cannot be bypassed by IP rotation (tied to authenticated user_id).
CORS	next.config.ts: restrict API routes to same-origin only. No wildcard CORS headers.
Content Security Policy	Add CSP headers in next.config.ts: restrict script-src, connect-src. No unsafe-inline in production.
Error Exposure	API routes never return raw error stack traces to the client. Log full errors to Sentry. Return generic messages to client.
HTTPS Only	next.config.ts: redirect all HTTP to HTTPS in production. Enforced via Vercel config.
Resume Data Warning	Add a server-side log whenever a resume is deleted, noting the user_id and timestamp (for audit purposes).

9. Testing Requirements — Complete Spec

Tests are written alongside feature code, not after. Every API route, utility function, and critical UI flow must have tests before that phase is marked complete.

9.1 Unit Tests (Vitest) — tests/unit/

Test File	What It Tests
resume-parser.test.ts	pdf-parse: valid PDF, scanned PDF (< 200 chars), password-protected PDF rejection, oversized file rejection. mammoth: valid DOCX, empty DOCX, corrupted file handling.
gemini-prompt.test.ts	buildSystemPrompt() returns string with all required sections. buildUserPrompt() correctly interpolates all variables. PROMPT_VERSION constant exists. Prompt contains all 13 rules.
zod-schemas.test.ts	TailoredResumeSchema: valid input passes, missing required fields fail, ATS score out of range fails, URL fields validate correctly. All API input schemas validated with valid and invalid inputs.

ats-scorer.test.ts	Keyword extraction from JD returns expected array. Score calculation: 10/10 = 100, 0/10 = 0, 7/10 = 70. Arabic keyword matching works correctly.
rate-limiter.test.ts	First 5 requests succeed. 6th request returns RATE_LIMIT error. Different user_ids have independent limits. Mocked Upstash client.
sanitize.test.ts	Null bytes stripped. Control characters removed. Script tags removed. Max length enforced. Injection patterns detected and flagged.

9.2 Integration Tests (Vitest + MSW) — tests/integration/

Test File	What It Tests
api-tailor.test.ts	Full pipeline with mocked Gemini response: returns valid TailoredResume. Invalid Gemini response triggers retry. Rate limit exceeded returns 429. Unauthenticated request returns 401. Saves to DB.
api-parse.test.ts	Valid PDF returns text. Scanned PDF returns SCANNED_PDF error. File > 5MB returns 413. Non-PDF returns 400. Unauthenticated returns 401.
api-export.test.ts	Generates PDF for valid resumeId. Returns 403 if resumeId belongs to different user. Returns 404 for unknown resumeId. Uploads to Storage and returns signed URL.
api-email.test.ts	Sends email with PDF attachment. Marks email_sent=true. Re-send works even if already sent. Resend SDK errors return 502 with generic message.
supabase-rls.test.ts	User A cannot SELECT user B's resumes. User A cannot INSERT into user B's resumes. User A cannot access user B's files in Storage. All 4 RLS operations verified per table.

9.3 End-to-End Tests (Playwright) — tests/e2e/

Test File	Critical Flows Covered
auth.spec.ts	Sign up with valid email → email verification flow → login → session persists on reload → logout clears session → unauthenticated redirect works.
tailoring-flow.spec.ts	Upload PDF → paste job description → click Tailor → AI processing state renders → result card appears with ATS score → Keywords tab shows matched/missing → Download PDF works → Send Email button works.
tailoring-flow.spec.ts (text)	Paste text resume (no file) → paste job description → tailor → same assertions as file flow.

history.spec.ts	Tailored resume appears in History list → click opens detail page → all 3 tabs render → download from detail page works → search filters results.
settings.spec.ts	Language switch EN → AR: sidebar flips RTL, content switches to Arabic → toggle email notifications → save shows toast → change full name → Delete Account flow shows confirmation dialog.
rtl.spec.ts	Switch to Arabic → sidebar is on right → all flex layouts are mirrored → text-align is right → ATS score ring renders → job description textarea accepts Arabic input.

9.4 Test Configuration Files

```
// tests/setup/vitest.config.ts
import { defineConfig } from 'vitest/config';
import { resolve } from 'path';

export default defineConfig({
  test: {
    environment: 'node',
    globals: true,
    setupFiles: ['./tests/setup/test-utils.tsx'],
    coverage: {
      provider: 'v8',
      reporter: ['text', 'json', 'html'],
      thresholds: { lines: 80, functions: 80, branches: 75 },
    },
  },
  resolve: { alias: { '@': resolve(__dirname, './src') } },
});

// tests/setup/playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests/e2e',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  reporter: [['html', { outputFolder: 'reports/playwright' }]],
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'Mobile Safari', use: { ...devices['iPhone 14'] } },
  ],
  webServer: { command: 'npm run dev', url: 'http://localhost:3000',
reuseExistingServer: !process.env.CI },
});
```

10. Phase-by-Phase Build Plan

Execute phases strictly in order. Each phase ends with a written report to /reports/phase-N-report.md. Do not mark a phase complete until all tasks and tests in that phase pass.

PHASE 0

Foundation & Setup

Project scaffolding, environment, CI, database, documentation skeleton

Tasks

1. Create the Next.js 14 project: npx create-next-app@latest vigilante-resume-tailor --typescript --tailwind --app --src-dir
2. Configure tsconfig.json: enable strict, noImplicitAny, strictNullChecks. Set @/* path alias.
3. Install all dependencies (see package.json spec in Section 2). Run npm audit — fix any high/critical vulnerabilities.
4. Create .gitignore (include .env.local, .env, node_modules, .next, coverage, playwright-report).
5. Create .env.local and .env.example with all keys from Section 4. Output ACTION REQUIRED block for user to fill values.
6. Set up ESLint (eslint-config-next + custom rules) and Prettier. Add pre-commit hooks via husky + lint-staged.
7. Set up GitHub Actions CI: .github/workflows/ci.yml — runs on push/PR to main: npm run lint, npm run type-check, npm run test:unit.
8. Initialize Supabase client files: src/lib/supabase/client.ts, server.ts, middleware.ts.
9. Output Supabase setup instructions (Section 4.2) as ACTION REQUIRED. Wait for user confirmation.
10. Run migrations 001, 002, 003 via Supabase dashboard SQL editor or Supabase CLI.
11. Verify RLS: run manual cross-user test query in Supabase SQL editor. Document result in phase report.
12. Create /docs/requirements.md — write a full requirements document based on this build prompt.
13. Create all placeholder report files in /reports/ for phases 0–6.
14. Create folder skeleton: /tests/unit, /tests/integration, /tests/e2e, /tests/setup.
15. Install and configure Sentry: run npx @sentry/wizard@latest -i nextjs.
16. Install and configure Posthog: add PosthogProvider to app/layout.tsx.
17. Write tests/setup/vitest.config.ts and tests/setup/playwright.config.ts.
18. Write tests/setup/test-utils.tsx: custom render wrapper with ThemeProvider, next-intl, QueryClient.

Phase 0 Completion Criteria

- npm run lint — passes with 0 errors

- npm run type-check — passes with 0 errors
- npm run test:unit — passes (only setup tests exist at this point)
- Supabase project created, all 3 migrations applied, RLS verified
- /docs/requirements.md written and committed
- /reports/phase-0-report.md written (project setup summary, all deps, Supabase config status, any issues)

PHASE 1

Core Infrastructure

Auth, middleware, i18n, design system, base layout

Tasks

19. Implement Supabase Auth: sign up, sign in, sign out, email verification callback route (/auth/callback/route.ts).
20. Implement src/middleware.ts: session refresh on all routes, redirect /dashboard/* to /auth/login if unauthenticated.
21. Install shadcn/ui: npx shadcn@latest init. Add components: button, input, textarea, card, dialog, switch, select, tabs, badge, separator, tooltip, avatar, dropdown-menu.
22. Apply the full design system from globals.css: all CSS tokens (colors, spacing, radii, shadows, fonts). Import DM Sans, Bebas Neue, JetBrains Mono.
23. Implement next-themes: ThemeProvider in layout.tsx. Dark mode as default.
24. Implement next-intl: set up i18nConfig, messages/en.json and messages/ar.json with all keys from Section 9 of the UI Design Prompt. Middleware for locale detection.
25. Build the RTL system: useLanguage hook reads from profiles.language. Sets dir attribute on <html>. Applies DM Sans + Noto Sans Arabic fallback. Uses logical CSS throughout.
26. Build the App Shell (dashboard/layout.tsx): Sidebar component, header area, content slot.
27. Build all components listed in /components/app/: Sidebar, ATSScoreRing, KeywordTags, FileUploadZone, ResumePreview, DiffView, AIProcessingState, HistoryCard, ResultCard, LanguageToggle, TailoringStrengthControl.
28. Build Landing page (app/page.tsx): hero, feature strip, how-it-works, footer — per UI Design Prompt Section 4.1.
29. Build Auth pages: login and signup — per UI Design Prompt Section 4.2.
30. Configure Sentry error boundaries on all pages.

Tests to Write in Phase 1

- tests/unit/zod-schemas.test.ts — write all schema tests (they'll fail until schemas exist; write them now).
- tests/e2e/auth.spec.ts — write auth E2E tests.

Phase 1 Completion Criteria

- Authentication works end-to-end: sign up, email confirm, login, logout.
- Middleware correctly protects /dashboard/* routes.
- Language toggle switches EN ↔ AR with RTL flip — verified on Landing and Sidebar.
- Dark/light mode toggle works.

- npm run test:unit — passes. npm run test:e2e -- tests/e2e/auth.spec.ts — passes.
- /reports/phase-1-report.md written.

PHASE 2

File Parsing & AI Pipeline

Resume parsing, Gemini integration, tailoring API, rate limiting

Tasks

31. Install file-type library. Implement src/lib/parser/pdf-parser.ts and docx-parser.ts with all validations from Section 7.1.
32. Implement /api/parse-resume/route.ts per Section 7.1 spec.
33. Implement src/lib/scraper/job-scraper.ts with cheerio + timeout protection.
34. Implement /api/fetch-job/route.ts per Section 7.2 spec including prompt injection guard.
35. Implement src/lib/security/sanitize.ts: null byte stripping, control char removal, injection pattern detection, max length enforcement.
36. Implement src/lib/gemini/client.ts, prompt.ts (PROMPT_VERSION = '1.0.0'), schema.ts.
37. Implement src/lib/ats/scorer.ts: keyword extraction from JD, keyword matching against tailored resume, score calculation.
38. Implement src/lib/rate-limit/limiter.ts using @upstash/ratelimit. Per-user-id sliding window: 5 requests/day.
39. Implement /api/tailor/route.ts per Section 7.3 spec: full pipeline, retry logic, DB write, Storage upload.
40. Add 60s function timeout to next.config.ts for /api/tailor.
41. Implement src/hooks/useTailoring.ts: React Query mutation wrapping /api/tailor + /api/parse-resume. Manages pipeline state (idle, parsing, analyzing, tailoring, generating, success, error).

Tests to Write in Phase 2

- tests/unit/resume-parser.test.ts — all parsing tests with mock files.
- tests/unit/gemini-prompt.test.ts — prompt builder tests.
- tests/unit/zod-schemas.test.ts — complete all schema tests.
- tests/unit/ats-scorer.test.ts — keyword extraction and scoring.
- tests/unit/rate-limiter.test.ts — with mocked Upstash client.
- tests/unit/sanitize.test.ts — sanitization and injection detection.
- tests/integration/api-tailor.test.ts — with mocked Gemini (MSW or vi.mock).
- tests/integration/api-parse.test.ts

Phase 2 Completion Criteria

- All unit tests pass (100% of Phase 2 test files).
- Integration tests pass for /api/tailor and /api/parse-resume with mocked Gemini.
- Manual test: upload a real PDF + paste a real job description → Gemini returns valid JSON → DB record created.
- Rate limit: 6th request returns 429. Confirmed in manual test.
- /reports/phase-2-report.md written.

PHASE 3

PDF Generation & Email Delivery

3 PDF templates, Supabase Storage, Resend email integration

Tasks

42. Implement all 3 PDF templates in src/lib/pdf/templates/: classic.tsx (ATS-safe, single column, clean), modern.tsx (slight color accents, section headers), arabic-rtl.tsx (full RTL, Arabic typography, right-anchored layout).
43. Implement src/lib/pdf/generator.ts: accepts TailoredResume + template selection → returns PDF buffer. Each template renders all sections: contact, summary, experience, education, skills, certifications.
44. Ensure all templates pass basic ATS rules: no images, no tables for layout, no multi-column (classic), single-pass readable text. Use only standard PDF fonts (Helvetica, Times New Roman) for ATS compatibility. DM Sans/Bebas Neue are for screen only.
45. Implement /api/export-pdf/route.ts per Section 7.4: fetch from DB, generate PDF, upload to Storage, return signed URL.
46. Implement src/lib/email/sender.ts: Resend SDK wrapper. Branded HTML email template. PDF attachment as buffer.
47. Implement /api/send-email/route.ts per Section 7.5 spec.
48. Output Resend setup ACTION REQUIRED block. Wait for user confirmation of DNS records.

Tests to Write in Phase 3

- tests/unit/pdf-generator.test.ts — render each template with mock TailoredResume, verify buffer is not empty, verify key fields appear in output.
- tests/integration/api-export.test.ts
- tests/integration/api-email.test.ts — with mocked Resend SDK.

Phase 3 Completion Criteria

- All 3 PDF templates render without error with sample resume data.
- Classic template is verified ATS-safe (text-only, single column, no images).
- Arabic RTL template renders correct RTL layout.
- Email delivery works: test email received with PDF attachment.
- All unit and integration tests for Phase 3 pass.
- /reports/phase-3-report.md written.

PHASE 4

Dashboard & Full UI Integration

Connect all backend APIs to the frontend, complete all pages

Tasks

49. Build Dashboard page (dashboard/page.tsx): wire FileUploadZone → /api/parse-resume, job input with URL import → /api/fetch-job, TailoringStrengthControl + template selector, CTA → useTailoring mutation, AIProcessingState for loading, ResultCard for success with all 3 tabs (Preview, Keywords, Changes/Diff).
50. Build DiffView component: compares originalText against tailored resume sections, highlights removed (red strikethrough) and added (teal highlight) text diff.
51. Build History page: React Query for fetching paginated resumes list. Search + filter UI. HistoryCard components with hover states. Load More pagination.
52. Build History Detail page [id]: full ResultCard view. Back button. Metadata bar. All download/email actions.
53. Build Settings page: all 3 cards (Language, Notifications, Account). Inline save per card. Language toggle with instant RTL switch. Delete Account dialog with confirmation. Test email button.
54. Wire all Posthog events: tailoring_started, tailoring_completed (with ATS score), pdf_downloaded, email_sent, language_switched, template_selected.
55. Implement empty states for History page and result tabs.
56. Implement all error states: upload error, parse error, Gemini error, rate limit error, email error — each with proper copy, icon, and recovery action.
57. Write /docs/walkthrough.md: step-by-step product walkthrough for a new user.

Tests to Write in Phase 4

- tests/e2e/tailoring-flow.spec.ts — full happy path.
- tests/e2e/history.spec.ts
- tests/e2e/settings.spec.ts
- tests/e2e/rtl.spec.ts

Phase 4 Completion Criteria

- Full end-to-end tailoring flow works in browser (upload → tailor → preview → download → email).
- History page shows past resumes with correct data.
- Settings language toggle switches RTL instantly.
- All 4 E2E test suites pass on Chromium and Mobile Safari.
- /docs/walkthrough.md written and committed.
- /reports/phase-4-report.md written.

PHASE 5

Security Hardening & QA

Full security audit, accessibility, performance, all tests passing

Security Audit Checklist — All Must Pass

58. Verify all items in Section 8 Security Requirements table are implemented. Check each one manually.
59. Run npm audit — 0 high or critical vulnerabilities allowed.
60. Verify .env.local is in .gitignore. Check git log to ensure no secrets were ever committed. If any were: rotate all affected keys immediately.

61. Verify server-side-only modules: grep the entire codebase for SUPABASE_SERVICE_ROLE_KEY, GEMINI_API_KEY, RESEND_API_KEY — they must NEVER appear in /src/app/ client components or be prefixed NEXT_PUBLIC_.
62. Run tests/integration/supabase-rls.test.ts — all RLS tests must pass.
63. Test file upload: attempt to upload a .exe, .js, .php, .svg file — all must be rejected with 400.
64. Test prompt injection: attempt to inject 'Ignore all previous instructions' in job description URL — must be caught by INJECTION_DETECTED guard.
65. Verify Content-Security-Policy headers are set in next.config.ts.
66. Verify all Storage URLs are signed (not public). Attempt to access a file URL after expiry — must return 403.

Accessibility Audit — All Must Pass

67. Run axe-core audit on all 5 pages (Playwright + @axe-core/playwright). 0 critical accessibility violations allowed.
68. Manual keyboard navigation test: Tab through entire Dashboard without a mouse. All actions must be reachable.
69. Verify focus rings are visible on all interactive elements in both dark and light mode.
70. Verify all ARIA labels specified in the UI Design Prompt are present.
71. Verify screen reader announces AI processing steps via aria-live='polite'.

Performance Audit

72. Run Lighthouse on Landing page and Dashboard: score ≥ 85 on Performance, ≥ 90 on Accessibility, ≥ 90 on Best Practices, ≥ 90 on SEO.
73. Verify @react-pdf/renderer PDF generation doesn't block the main thread (runs in API route, server-side — already correct if implemented per spec).
74. Verify Gemini API call has proper streaming or timeout handling. No request should hang indefinitely.

Phase 5 Completion Criteria

- All security checklist items verified and documented.
- All RLS tests pass.
- 0 critical axe-core accessibility violations.
- Lighthouse scores meet minimums.
- npm run test (unit + integration + e2e) — ALL tests pass.
- /reports/phase-5-report.md written (full security audit results, accessibility results, Lighthouse scores).

PHASE 6

Deployment, Monitoring & Documentation

Vercel deployment, Sentry verification, Posthog, final docs

Tasks

75. Prepare for Vercel deployment: ensure next.config.ts is production-ready (HTTPS redirect, CSP headers, image domains). Verify all environment variables are set in Vercel project settings (not just .env.local).
76. Output ACTION REQUIRED: Vercel deployment steps. The user must: (a) connect GitHub repo to Vercel, (b) add all environment variables from .env.local into Vercel project settings, (c) update NEXT_PUBLIC_APP_URL to production domain, (d) update Supabase Auth Site URL and Redirect URLs to production domain.
77. After production deployment: verify Sentry is receiving errors (throw a test error, check Sentry dashboard).
78. Verify Posthog is receiving events (trigger a tailoring, check Posthog Live Events dashboard).
79. Update Resend sending domain DNS records to production domain if different from development.
80. Run all E2E tests against the production URL: playwright test --project=chromium.
81. Write /docs/final.md: complete final documentation covering architecture, all API routes, DB schema, environment variables reference, troubleshooting guide, known limitations, and maintenance notes.
82. Final clean-up: remove any console.log statements, commented-out code, TODO comments from production code. Run npm run lint — 0 errors.
83. Tag the release: git tag v1.0.0 and push.

Phase 6 Completion Criteria

- Application is live on Vercel production URL.
- All E2E tests pass against production URL.
- Sentry receiving events. Posthog receiving events.
- Email delivery working in production (received test email from production sender).
- /docs/final.md written and committed.
- /reports/phase-6-report.md written (deployment summary, production test results, monitoring status, any outstanding issues).
- README.md updated with: project description, tech stack, setup instructions, environment variables reference.

11. Phase Report Template

Each phase report must follow this exact structure. Written in Markdown to /reports/phase-N-report.md.

```
# Phase N Report — [Phase Title]
**Date:** YYYY-MM-DD
**Status:** Complete | Blocked | Partial

## Summary
2-3 sentences summarizing what was accomplished in this phase.

## Tasks Completed
- [ ] Task 1 (with any implementation notes)
```

```
- [ ] Task 2
...

## Tests Written & Status
| Test File | Tests | Passing | Failing | Skipped |
|-----------|-------|---------|---------|---------|
| file.test.ts | 12 | 12 | 0 | 0 |

## Issues Encountered
List any blockers, unexpected behaviors, or deviations from spec.
For each issue: describe it, how it was resolved (or why it's still open).

## Deviations from Spec
If ANY part of this phase was implemented differently from the spec, document
it here.
No undocumented deviations allowed.

## Security Notes
Any security-relevant decisions made this phase.

## Next Phase Readiness
Confirm: all completion criteria met. List any blockers for the next phase.
```

12. Documentation Requirements

12.1 docs/requirements.md — Written at Phase 0 Start

- Full functional requirements list (all features, all pages, all API routes).
- Non-functional requirements (security, performance, accessibility, RTL, i18n).
- Tech stack rationale — why each library was chosen over alternatives.
- Data model description (all tables, columns, relationships).
- External dependencies and their purpose.
- Scope exclusions (what is NOT being built).

12.2 docs/walkthrough.md — Written at Phase 4 End

- Step-by-step user journey from landing page to downloading a tailored resume.
- Each step has: a description, what the user sees, what happens in the background.
- Screenshots not required — describe each state in words clearly enough that a non-technical user can follow.
- Include: sign up flow, first resume tailoring, reading the ATS score and keywords, downloading PDF, email delivery, history access, settings (language, email toggle).
- Arabic user flow: how to switch to Arabic and tailor an Arabic resume.

12.3 docs/final.md — Written at Phase 6 End

- Architecture overview: how all layers connect (frontend → API routes → Gemini → Supabase → Resend).
- All API routes: method, path, auth requirement, input schema, output schema, error codes.
- Database schema: all tables, all columns, types, constraints, RLS policies.
- Environment variables: name, description, where to get it, whether it's public or server-only.
- Deployment guide: Vercel configuration, required env vars, Supabase production setup.
- Troubleshooting: top 10 most likely issues with diagnosis and resolution steps.
- Known limitations: Gemini rate limits, Vercel function timeout, PDF ATS caveats, Arabic text rendering edge cases.
- Maintenance notes: how to update the Gemini prompt version, how to add a new PDF template, how to expand language support.

13. Final Instructions to the Agent

FINAL DIRECTIVE — Read This Last

You now have a complete, unambiguous specification. Here is the exact order of operations:

1. Read this entire document before writing any code.
2. Confirm you understand the full scope by outputting a brief summary of all 7 phases.
3. Begin Phase 0. Create the project. Create .env.local. Output all ACTION REQUIRED blocks for external services.
4. Pause after each ACTION REQUIRED block and wait for the user to confirm completion.
5. At the end of each phase, write the phase report to /reports/phase-N-report.md.
6. Before starting the next phase, explicitly state: 'Phase N complete. All criteria met. Starting Phase N+1.'
7. If you encounter a blocker, output a BLOCKED block describing: what is blocked, why, and what the user needs to do to unblock it.
8. Never invent features, routes, or components not specified in this document.
9. Never skip tests. Never mark a phase complete with failing tests.
10. The design system from the UI Design Prompt is already implemented. Do not re-implement or modify it — wire it up.

The goal is a production-grade application that is secure, tested, documented, and ready for real users.
Build it like it ships. Because it does.